# Kilwater API - dokumentacja

Endpoint publiczny: `POST /api/zapytanie` (formularz zapytania o rejs)
Pliki pomocnicze: `/sitemap.xml`, `/robots.txt`

```
https://kilwater.com
http://localhost:4520   (lokalnie, npm run dev)
```

### Spis treści


1. Autoryzacja i limity
2. Konwencje i błędy
3. POST /api/zapytanie
4. Wartości pól wyboru i przykład
5. Odpowiedzi
6. Wysyłka z własnej strony
7. Dostarczanie zapytań
8. GET /sitemap.xml i robots.txt
9. Dane osobowe
10. Lista kontrolna przed wdrożeniem
11. Rozwiązywanie problemów


## Autoryzacja i limity

API nie wymaga kluczy, ciasteczek ani sesji. Jedyny endpoint, `POST /api/zapytanie`, przyjmuje zapytania z formularza na stronie głównej i z dowolnego innego źródła, które wyśle poprawny JSON. Ochronę przed nadużyciami zapewniają limity i walidacja opisane niżej.

| Zabezpieczenie | Opis |
|---|---|
| Limit żądań | **5 żądań na 10 minut z jednego adresu IP**. Szóste i kolejne dostają `429`. Limit jest liczony w pamięci procesu serwera i zeruje się przy restarcie. |
| Rozmiar treści | Maksymalnie 10 000 bajtów. Większa treść dostaje `413`. |
| Typ treści | Wymagany nagłówek `Content-Type: application/json`. Inny typ dostaje `415`. |
| Pułapka na boty | Pole `website` musi być puste. Wypełnione oznacza bota: serwer odpowiada `200`, ale zapytania nie przetwarza. |
| Lista dozwolonych pól | Serwer przyjmuje tylko pola opisane w tej dokumentacji. Każde inne pole, np. `isAdmin` albo `role`, kończy się błędem `400` z nazwą tego pola. |

### Przykład żądania (curl)

```
curl -X POST https://kilwater.com/api/zapytanie \
  -H 'Content-Type: application/json' \
  -d '{"destination":"chorwacja","startDate":"2027-06-12","nights":"7",
       "guests":6,"cruiseType":"ze-skipperem","extras":["sup"],
       "name":"Anna Kowalska","email":"anna@example.com","phone":"",
       "message":"","consent":true,"website":""}'
```

### Kolejność sprawdzania

1. Typ treści ( `415` ).
2. Rozmiar treści ( `413` ).
3. Limit żądań z adresu IP ( `429` ).
4. Poprawność JSON ( `400` bez pola `errors` ).
5. Pułapka na boty ( `200` bez przetwarzania).
6. Walidacja pól ( `400` z polem `errors` ).
7. Wysyłka e-mail ( `200` albo `503` ).

Żądania odrzucone na krokach 1 i 2 nie zużywają limitu z kroku 3.

## Konwencje i błędy

| Zasada | Opis |
|---|---|
| Format | Odpowiedzi to JSON w UTF-8 z nagłówkiem `Cache-Control: no-store`. |
| Daty | Dzień bez godziny w formacie `YYYY-MM-DD`, np. `2027-06-12`. Porównanie z dniem dzisiejszym w UTC. |
| Wartości wyboru | Akwen, długość, rodzaj rejsu i dodatki przyjmują stałe identyfikatory (małe litery, myślniki). Pełna lista w sekcji „Wartości pól wyboru”. |
| Teksty | Spacje na początku i końcu pól tekstowych są usuwane przed walidacją. |
| Język | Komunikaty błędów są po polsku i mówią, co poprawić. Formularz pokazuje je bez zmian pod polami. |

Odpowiedź zawsze ma pole `ok`. Pole `errors` pojawia się tylko przy błędach walidacji:

```
{"ok": false, "errors": {"email": "Podaj poprawny adres e-mail, np.
jan@przyklad.pl."}}
```

| Kod | Kiedy |
|---|---|
| `200` | Zapytanie przyjęte i wysłane (albo zapisane w logu w trybie testowym). |
| `400` | Treść nie jest poprawnym JSON-em ( `{"ok": false}` ) albo walidacja nie przeszła ( `errors` z kluczem = nazwa pola). |
| `404` | Inna metoda niż `POST` , np. `GET /api/zapytanie` . |
| `413` | Treść żądania większa niż 10 000 bajtów. |
| `415` | Nagłówek `Content-Type` inny niż `application/json` . |
| `429` | Przekroczony limit 5 żądań na 10 minut z jednego IP. |
| `503` | Wysyłka e-mail nie jest skonfigurowana na produkcji albo dostawca poczty zwrócił błąd. Szczegóły tylko w logu serwera. |

## POST /api/zapytanie

`POST` `/api/zapytanie`

Przyjmuje zapytanie o rejs, sprawdza je i przekazuje brokerowi e-mailem z adresem klienta w polu Reply-To.

### Treść żądania (JSON)

| Parametr | Wymagany | Opis |
|---|---|---|
| `destination` | tak | Akwen. Jedna z wartości w sekcji „Wartości pól wyboru”. |
| `startDate` | tak | Data wypłynięcia `YYYY-MM-DD` , dziś albo później. |
| `nights` | tak | Długość rejsu jako tekst, np. `"7"` . |
| `guests` | tak | Liczba osób, liczba całkowita od 1 do 12. Przyjmuje też liczbę zapisaną jako tekst. |
| `cruiseType` | tak | Rodzaj rejsu. |
| `extras` | nie | Tablica dodatków, domyślnie pusta. Każdy dodatek najwyżej raz. |
| `name` | tak | Imię i nazwisko, od 2 do 120 znaków. |

| Parametr | Wymagany | Opis |
| --- | --- | --- |
| `email` | tak | Adres e-mail, maksymalnie 200 znaków. Trafia do pola Reply-To wiadomości dla brokera. |
| `phone` | nie | Telefon, np. `+48 600 000 000`: cyfry, spacje, nawiasy i myślniki, co najmniej 7 znaków. Puste dozwolone. |
| `message` | nie | Wiadomość, maksymalnie 1000 znaków. |
| `consent` | tak | Musi mieć wartość `true`: akceptacja regulaminu i informacji z polityki prywatności. |
| `website` | nie | Pułapka na boty, zawsze puste. Patrz „Autoryzacja i limity". |

## Wartości pól wyboru i przykład

Pola wyboru przyjmują tylko poniższe identyfikatory. Inna wartość kończy się błędem `400` z komunikatem dla tego pola.

### `destination` - akwen

| Wartość | Znaczenie |
| --- | --- |
| `chorwacja` | Chorwacja: Dalmacja i Kornati |
| `grecja` | Grecja: Cyklady i Wyspy Jońskie |
| `wlochy` | Włochy: Wybrzeże Amalfi, Sardynia, Sycylia |
| `mazury` | Mazury: Wielkie Jeziora Mazurskie |
| `nie-wiem` | Jeszcze nie wiem, akwen do ustalenia z brokerem |

### `nights` - długość rejsu

| Wartość | Znaczenie |
| --- | --- |
| `3`, `5`, `7`, `10`, `14` | Liczba nocy. Formularz domyślnie zaznacza `7`. |
| `inna` | Inna długość, do ustalenia w rozmowie |

### `cruiseType` - rodzaj rejsu

| Wartość | Znaczenie |
|---|---|
| `pelna-zaloga` | Z pełną załogą: skipper, hostessa i kucharz |
| `ze-skipperem` | Ze skipperem, bez reszty załogi |
| `bez-zalogi` | Bez załogi, na własne uprawnienia klienta |

### `extras` - dodatki

| Wartość | Znaczenie |
|---|---|
| `sup` | Deski SUP |
| `snorkeling` | Sprzęt do snorkelingu |
| `ponton` | Ponton z silnikiem |
| `transfer` | Transfer z lotniska do mariny |
| `zaopatrzenie` | Zaopatrzenie jachtu przed przyjazdem |
| `posciel` | Pościel, ręczniki i sprzątanie po rejsie |

### Przykład żądania

Treść żądania `POST` z nagłówkiem `Content-Type: application/json`, ze wszystkimi polami:

```
{
  "destination": "chorwacja",
  "startDate": "2027-06-12",
  "nights": "7",
  "guests": 6,
  "cruiseType": "ze-skipperem",
  "extras": ["sup", "transfer"],
  "name": "Anna Kowalska",
  "email": "anna@example.com",
  "phone": "+48 600 000 000",
  "message": "Pierwszy rejs, dwoje dzieci.",
  "consent": true,
  "website": ""
}
```

## Odpowiedzi

### Odpowiedź - błąd walidacji (400)

```
{
  "ok": false,
  "errors": {
    "destination": "Wybierz akwen albo „Jeszcze nie wiem”.",
    "startDate": "Data wypłynięcia nie może być w przeszłości.",
    "guests": "Maksymalnie 12 osób na jednym jachcie.",
    "name": "Podaj imię i nazwisko.",
    "email": "Podaj poprawny adres e-mail, np. jan@przyklad.pl.",
    "phone": "Podaj numer w formacie +48 600 000 000 albo zostaw puste.",
    "consent": "Zaznacz akceptację regulaminu, żebyśmy mogli przygotować ofertę."
  }
}
```

### Odpowiedź - niedozwolone pola (400)

```
{
  "ok": false,
  "errors": {
    "isAdmin": "To pole nie jest obsługiwane.",
    "role": "To pole nie jest obsługiwane."
  }
}
```

### Odpowiedź - pozostałe błędy (400, 413, 415, 429, 503)

```
{"ok": false}
```

### Odpowiedź - sukces (200)

```
{"ok": true}
```

### Znaczenie kluczy w odpowiedzi

| Klucz | Opis |
|---|---|
| `ok` | `true`, gdy zapytanie zostało przyjęte. `false` przy każdym błędzie. |
| `errors` | Tylko przy błędzie walidacji. Obiekt: nazwa pola → komunikat po polsku z informacją, co poprawić. Pola, których serwer nie obsługuje, dostają komunikat „To pole nie jest obsługiwane.”. |

Sukces nie zwraca identyfikatora zapytania, bo serwer niczego nie zapisuje w bazie. Zapytanie istnieje tylko jako e-mail u brokera.

## Wysyłka z własnej strony

Formularz na kilwater.com robi dokładnie to, co poniższy kod. Tak samo można wysłać zapytanie z innej strony albo aplikacji, np. z kampanii albo strony partnera.

```
const response = await fetch("https://kilwater.com/api/zapytanie", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(inquiry),
});
const result = await response.json().catch(() => ({}));

if (response.ok) showThankYou();
else if (response.status === 400 && result.errors) showFieldErrors(result.errors);
else if (response.status === 429) showMessage("Odczekaj kilka minut i spróbuj
ponownie.");
else showMessage("Nie udało się wysłać zapytania. Spróbuj ponownie za chwilę.");
```

| Sytuacja | Co pokazać użytkownikowi |
|---|---|
| `200` | Potwierdzenie i informację, że broker odezwie się na podany adres e-mail. |
| `400` z `errors` | Komunikat pod każdym polem z obiektu `errors` i fokus na pierwszym błędnym polu. |
| `429` | Prośbę o odczekanie kilku minut. Ponowne wysłanie od razu znowu dostanie `429`. |
| Inny błąd albo brak sieci | Ogólny komunikat z prośbą o ponowienie i kontaktem e-mail `kontakt@kilwater.com`. |

## Dostarczanie zapytań

Po walidacji serwer wysyła zapytanie przez API Resend jako zwykły tekst (bez HTML). Temat ma postać `Zapytanie o rejs: Chorwacja, 2027-06-12`, a pole Reply-To wskazuje adres klienta, więc broker odpowiada bezpośrednio z poczty.

| Zmienna | Opis |
|---|---|
| `RESEND_API_KEY` | Klucz API Resend. Tylko na serwerze, nigdy w przeglądarce. |
| `INQUIRY_TO_EMAIL` | Adres, na który przychodzą zapytania, np. `kontakt@kilwater.com`. |
| `INQUIRY_FROM_EMAIL` | Nadawca zweryfikowany w Resend. |
| `INQUIRY_DRY_RUN` | `1` = zapytanie tylko w logu, do testów. |

### Treść e-maila (tryb testowy wypisuje to samo w logu)

```
Akwen: Chorwacja
Data wypłynięcia: 2027-06-12
Długość: 7 nocy
Liczba osób: 6
Rodzaj rejsu: Ze skipperem
Dodatki: Deski SUP, Transfer z lotniska

Imię i nazwisko: Anna Kowalska
E-mail: anna@example.com
Telefon: +48 600 000 000

Wiadomość: Pierwszy rejs, dwoje dzieci.
```

### Zachowanie zależnie od konfiguracji

| Konfiguracja | Zachowanie |
|---|---|
| Wszystkie trzy zmienne Resend ustawione | Wysyłka przez Resend. Błąd dostawcy → `503`. |
| Brak zmiennych, `npm run dev` | Zapytanie w logu, odpowiedź `200`. |
| Brak zmiennych, `INQUIRY_DRY_RUN=1` | Zapytanie w logu, odpowiedź `200` (używane przez testy e2e). |
| Brak zmiennych na produkcji | Odpowiedź `503`, żeby zapytania nie ginęły po cichu. |

## GET /sitemap.xml i robots.txt

`GET` `/sitemap.xml`

Mapa strony dla wyszukiwarek, generowana przy budowaniu, bez parametrów. Adresy powstają z `site` w `astro.config.mjs`, a strony 404 w mapie nie ma.

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url><loc>https://kilwater.com/</loc></url>
  <url><loc>https://kilwater.com/polityka-prywatnosci</loc></url>
  <url><loc>https://kilwater.com/regulamin</loc></url>
</urlset>
```

Plik `/robots.txt` pozwala indeksować całą stronę poza `/api/` i wskazuje tę mapę:

```
User-agent: *
Allow: /
Disallow: /api/

Sitemap: https://kilwater.com/sitemap.xml
```

## Dane osobowe

Gdzie trafiają dane klienta. Podstawy prawne opisuje `/polityka-prywatnosci`.

| Dane | Gdzie trafiają | Jak długo |
|---|---|---|
| Wszystkie pola formularza | E-mail do brokera, wysłany przez Resend jako pośrednika. Serwer strony niczego nie zapisuje w bazie ani na dysku. | Zgodnie z polityką prywatności. |
| Adres IP | Pamięć procesu serwera, tylko do liczenia limitu żądań. | Do 10 minut, znika też przy restarcie serwera. |
| Pełna treść zapytania w logu | Log serwera, tylko w `npm run dev` i przy `INQUIRY_DRY_RUN=1`. | Do wyczyszczenia logu. |
| Dane w przeglądarce | Formularz nie zapisuje ciasteczek ani `localStorage`. | Brak przechowywania. |

### Zasady dla integracji

- Wysyłaj zapytanie tylko po akceptacji regulaminu przez klienta, czyli z `consent: true` ustawionym przez niego, a nie automatycznie.
- Nie dopisuj do wiadomości danych, których klient sam nie podał, np. z systemu CRM.
- Informacje o zdrowiu, np. alergie przy wyżywieniu, przekazuj tylko wtedy, gdy klient sam je wpisał.
- Nie zapisuj odpowiedzi API z danymi klienta w logach własnej aplikacji. Odpowiedź zawiera tylko `ok` i ewentualne komunikaty błędów.

### Prośby klientów o ich dane

Serwer strony nie przechowuje zapytań, więc każdą prośbę obsługuje się w skrzynce brokera:

| Prośba | Co zrobić |
|---|---|
| Dostęp do danych | Wyszukaj w skrzynce wiadomości z adresem klienta w Reply-To i prześlij mu ich treść. |
| Usunięcie danych | Usuń te wiadomości ze skrzynki i z kosza, a potem potwierdź usunięcie klientowi. |
| Sprostowanie | Odpowiedz na wiadomość z poprawionymi danymi, żeby historia zawierała aktualną wersję. |
| Termin odpowiedzi | Do miesiąca od otrzymania prośby (art. 12 ust. 3 RODO). |

## Lista kontrolna przed wdrożeniem

Endpoint działa od razu po buildzie, ale na produkcji bez tych kroków zapytania nie dotrą do brokera albo będą słabiej chronione.

| Krok | Jak sprawdzić |
|---|---|
| Domena zweryfikowana w Resend | Panel Resend pokazuje status „Verified" przy `kilwater.com`. |
| Zmienne Resend ustawione na serwerze | Testowe zapytanie z formularza przychodzi na `INQUIRY_TO_EMAIL`, a nie kończy się `503`. |
| `INQUIRY_DRY_RUN` wyłączone | Zmienna nie istnieje albo jest pusta w środowisku produkcyjnym. |
| Prawdziwy adres IP klienta | Za reverse proxy serwer musi widzieć adres klienta, inaczej wszyscy dzielą jeden limit 5 żądań. |

| Krok | Jak sprawdzić |
| --- | --- |
| Nagłówki bezpieczeństwa | Hosting ustawia CSP, HSTS, `Referrer-Policy`, `Permissions-Policy` i `frame-ancestors`. |
| Ochrona przed botami | Przy większym ruchu dodaj Cloudflare Turnstile do formularza i sprawdzanie tokenu w endpoincie. |
| Adres strony | `site` w `astro.config.mjs` wskazuje docelową domenę, a `/sitemap.xml` zwraca właściwe adresy. |

## Rozwiązywanie problemów

Najczęstsze objawy po wdrożeniu i ich przyczyny. Szczegóły błędów wysyłki są zawsze w logu serwera, nigdy w odpowiedzi API.

| Objaw | Przyczyna i rozwiązanie |
| --- | --- |
| Każde zapytanie kończy się `503` | Na serwerze brakuje jednej ze zmiennych Resend albo klucz jest nieważny. Log pokazuje `brak konfiguracji wysyłki e-mail` albo `błąd wysyłki e-mail` z kodem od Resend. |
| Klienci szybko dostają `429` | Serwer stoi za reverse proxy i widzi jeden adres IP dla wszystkich. Przekaż adres klienta z proxy do serwera, żeby limit liczył się osobno dla każdego. |
| Maile z zapytaniami trafiają do spamu | Domena nadawcy nie ma rekordów SPF i DKIM z panelu Resend. Dodaj je w DNS i poczekaj na status „Verified”. |
| Data wypłynięcia „w przeszłości” dla dzisiejszej daty | Serwer porównuje daty w UTC, więc tuż po północy czasu polskiego dzisiejsza data bywa dla niego jeszcze jutrem. |
| Formularz pokazuje „Brak połączenia z internetem” | Żądanie nie dotarło do serwera, np. przez blokadę sieci albo wyłączony serwer. |